Final Report

Google drive link:
https://drive.google.com/drive/folders/1tqrJAStH3KUpY0v3oovARx_HKmDlwFcu?usp=drive_link

Title

Wage Related Data in the USA

Team Details

Team Members:

1 Doris Xia

2 Tianqi Qiu

Background and Skills:

Member 1: Doris Xia

Doris is a first-year graduate student pursuing a Master's in Applied Data Science and recently graduated from Rutgers University–New Brunswick with a bachelor's degree in Business Analytics and Information Technology. They have experience with programming and data analysis, specifically using Python for data retrieval, analysis, and visualization. They have a working knowledge of SQL, having worked with it to query and manage relational databases. Additionally, they have experience using R for data analysis and visualization. This group member's background includes working with machine learning techniques like regression, clustering, and neural networks.

Member 2: Tianqi Qiu

Tianqi Qiu is a first-year Master's student in Applied Data Science. He earned his Bachelor's degree in Business Analytics from the University of Rochester, where he gained hands-on experience with data analysis, statistical methods, and machine learning techniques. He is into using Python for data analysis, focusing on data processing, cleaning, and visualization. He has experience working with SQL databases and Tableau for visualizing data insights. Additionally, he is proficient in CSS and HTML, using these skills to build web-based data visualizations.

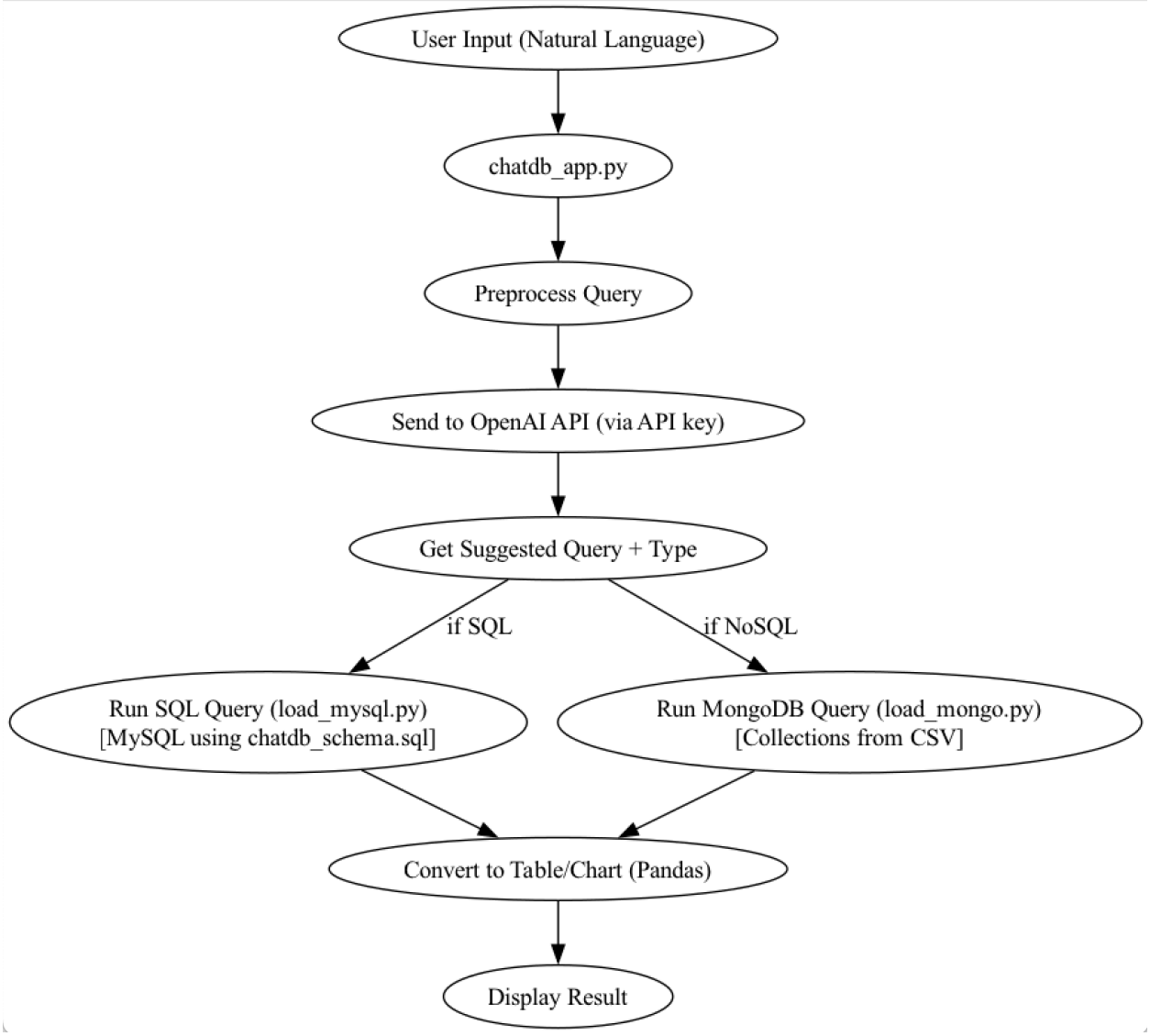
Introduction

ChatDB is a natural language interface that enables users to interact with both SQL and NoSQL databases seamlessly. Our project focuses on exploring wage-related data in the United States through natural language queries. Users can ask questions such as "What is the average wage for men with a college degree in 2020?" and the system automatically determines the appropriate database—MySQL or MongoDB—based on the context of the question. It then translates the query using GPT-4 into an executable SQL or PyMongo command and returns the corresponding data.

Planned Implementation

Our proposal outlined a unified interface for natural language interaction with MySQL and MongoDB databases. We planned to create normalized schemas in MySQL and flexible collections in MongoDB, use OpenAI GPT-4 to translate natural language into SQL and PyMongo syntax, support query types including SELECT, WHERE, GROUP BY, aggregation, joins, and data modifications (insert, update, delete), and build a minimal frontend (optional, later deprioritized). Over time, we simplified MySQL schema design, embedded demographic info in MongoDB documents, and relied fully on GPT-4 for query translation.

Architecture Design



The system follows a multi-step architecture. First, the user inputs a natural language question into the terminal. The system then scores keywords using a backend detection function to determine whether the query should be routed to MySQL or MongoDB. Based on this classification, a system prompt tailored to the target database is constructed and sent—along with the user query—to GPT-4. The response is parsed to extract the executable command. For SQL, the result is passed to pymysql for execution; for MongoDB, the command is parsed and securely evaluated using ast.literal_eval() before execution via pymongo. Output is formatted using tabulate (SQL) or pprint (MongoDB) and displayed to the user. This architecture abstracts the

complexities of query composition and schema knowledge, providing a seamless user experience.

An essential part of this architecture is prompt design. We crafted separate system prompts for MySQL and MongoDB backends, each tailored to the expected syntax and logic of the target database. The MySQL prompt emphasized relational concepts like SELECT, JOIN, and GROUP BY, while the MongoDB prompt focused on stages such as $match, $group, and $project. We iteratively refined the prompts with schema context and sample fields to improve model accuracy. This prompt engineering was critical to enabling GPT-4 to reliably produce valid and efficient queries across both systems.

Implementation

Functionalities: ChatDB includes backend detection using keyword scoring, GPT-4 query generation, SQL query execution using pymysql, and MongoDB query execution using pymongo. It supports SELECT, aggregation, projection, joins, and data modification operations. Schema exploration commands like SHOW TABLES and list collections are also supported. Additionally, MongoDB query arguments are safely evaluated using ast.literal_eval().

Tech Stack: The tech stack includes Python as the primary language, OpenAI's GPT-4 model for query translation, and the pymysql and pymongo libraries for database access. Data was ingested using pandas, and results were formatted using tabulate (for SQL) and pprint (for MongoDB). Both MySQL and MongoDB were hosted locally to allow fast and controlled experimentation.

The screenshot below demonstrates the successful startup of chatdb_app.py, confirming connections to both MySQL and MongoDB.

```
tianqi0301@luolingdeMacBook-Pro chatdb % cd Desktop/chatdb
cd: no such file or directory: Desktop/chatdb
tianqi0301@luolingdeMacBook-Pro chatdb % python3 chatdb_app.py
Successfully connected to MySQL.
Successfully connected to MongoDB.

Welcome to ChatDB!
Ask questions about living wages, poverty stats (MySQL), or wages by education (Mongo
DB - overall, gender, race collections).
Type 'help' for examples or 'exit' to quit.

User: show tables

ChatDB (SQL Schema):
+-------------------------------+
| Tables_in_chatdb_relational   |
+===============================+
| City                          |
+-------------------------------+
| HouseholdType                 |
+-------------------------------+
| LivingWage                    |
+-------------------------------+
| PovertyGroup                  |
+-------------------------------+
| PovertyWageDistribution       |
+-------------------------------+
| YearlyPovertyThresholds       |
+-------------------------------+

User: show collections

ChatDB (Mongo Schema):
Collections in 'chatdb_nosql': wages_race, wages_overall, wages_gender
```

To validate system behavior, we tested a variety of natural language queries across both databases. For example, “What is the average wage for Hispanic men with a high school degree in 2020?” was routed to MongoDB and translated into an aggregation pipeline, while “Show the top 5 cities by population density” executed an SQL query in MySQL using ORDER BY and LIMIT. Other examples included listing education levels for Black women in 2021 (MongoDB projection) and inserting a new city into the database (MySQL INSERT). These examples demonstrate the system’s ability to handle both structured and semi-structured queries appropriately. In cases where queries fell outside the supported schema, ChatDB responded with informative fallback messages.

```
User: What is the average wage for Hispanic men with a high school degree in 2020?
--> Routing to MongoDB...

Generated MongoDB Query:
```python
wages_race.find_one({'year': 2020}, {'_id': 0, 'hispanic_men_high_school': 1})
```

ChatDB (Mongo Result):
{'hispanic_men_high_school': 23.33}
```

```
User: Show the top 5 cities by population density
--> Routing to MySQL...

Generated SQL Query:
```sql
SELECT city_name, density FROM City ORDER BY density DESC LIMIT 5;
```

ChatDB (SQL Result):
+---------------+-----------+
| city_name | density |
+===============+===========+
| New York | 29298 |
+---------------+-----------+
| Jersey City | 19894 |
+---------------+-----------+
| San Francisco | 18635 |
+---------------+-----------+
| Boston | 13989 |
+---------------+-----------+
| Newark | 12927 |
+---------------+-----------+
```

```
User: Add a new city called Arcadia in California, with a population of 200000 and a
density of 5000.
--> Routing to MySQL...

Generated SQL Query:
```sql
INSERT INTO City (city_name, state_name, population_2020, density) VALUES ('Arcadia',
 'California', 200000, 5000);
```

ChatDB (SQL Result):
Query OK. 1 rows affected.

User: Show the city of Arcadia
--> Routing to MySQL...

Generated SQL Query:
```sql
SELECT * FROM City WHERE city_name = 'Arcadia';
```

ChatDB (SQL Result):
+-----------+-------------+--------------+-------------------+-------------------+---
---------------+-----------+
| city_id | city_name | state_name | population_2020 | population_2010 | la
nd_area_sqmi | density |
+===========+=============+==============+===================+===================+===
===============+===========+
| 408 | Arcadia | California | 200000 | |
               | 5000 |
+-----------+-------------+--------------+-------------------+-------------------+---
---------------+-----------+
```

We also captured sample outputs to demonstrate functionality. A MongoDB query about average wages for women with some college in 2022 returned a clean JSON object with the correct wage. A MySQL query to list cities by population density yielded a tabulated list sorted in descending order. For unsupported queries like “What is the capital of the United States?”, the

system returned a clear notice indicating the query could not be processed. These results illustrate the effectiveness and robustness of ChatDB's natural language interface.

```
User: average wages for women with some college in 2022
--> Routing to MongoDB...

Generated MongoDB Query:
```python
wages_gender.find_one({'year': 2022}, {'_id': 0, 'women_some_college': 1})
```

ChatDB (Mongo Result):
{'women_some_college': 21.76}
```

```
User: What is the capital of the United States?
--> Routing to MySQL...

Generated SQL Query:
```sql
The database schema provided does not contain information about the capital of countr
ies. Therefore, it's not possible to write a SQL query to answer this question.
```

ChatDB (SQL Result):
SQL Execution Error: (1064, "You have an error in your SQL syntax; check the manual t
hat corresponds to your MySQL server version for the right syntax to use near 'The da
tabase schema provided does not contain information about the capital of c' at line 1
")
```

Code Snippets from chatdb_app.py

1. Backend Detection Logic

```
# Basic backend detection based on keywords
sql_keywords = {"poverty", "living", "household", "threshold"}
nosql_keywords = {"wage", "education", "gender", "race"}

if any(word in user_input.lower() for word in sql_keywords):
      db_type = "sql"
elif any(word in user_input.lower() for word in nosql_keywords):
      db_type = "mongo"
else:
      db_type = "unknown"
```

This snippet demonstrates how backend detection is implemented using simple keyword matching. If the input contains SQL-related terms, it routes the query to the SQL database; otherwise, to MongoDB.

2. System Prompt Construction

```
# Construct system prompt based on detected backend
if db_type == "sql":
      system_prompt = SQL_SYSTEM_PROMPT
elif db_type == "mongo":
      system_prompt = MONGO_SYSTEM_PROMPT
else:
      system_prompt = "You are a helpful assistant."
```

Based on the backend type, the appropriate system prompt is selected and sent to GPT-4. This ensures the model produces database-specific queries.

3. GPT-4 Query Generation and API Call

```
response = openai.ChatCompletion.create(
      model="gpt-4",
      messages=[
      {"role": "system", "content": system_prompt},
      {"role": "user", "content": user_input}
      ]
)
response_content = response["choices"][0]["message"]["content"]
```

This section shows how the user input and system prompt are packaged into a GPT-4 API request. The model returns a database-ready query as a response.

4. SQL Execution using pymysql

```
try:
      cursor = mysql_conn.cursor()
      cursor.execute(response_content)
      results = cursor.fetchall()
      headers = [i[0] for i in cursor.description]
      print(tabulate(results, headers=headers, tablefmt="grid"))
```

Here, the returned SQL query is executed using `pymysql`, and the results are formatted into a readable table using the `tabulate` library.

5. MongoDB Execution using ast.literal_eval()

```
parsed_query = ast.literal_eval(response_content)
collection_name = parsed_query.pop("collection")
collection = mongo_db[collection_name]
```

```
result = collection.find_one(parsed_query)
pprint.pprint(result)
```

To avoid using `eval()`, the MongoDB query is parsed using `ast.literal_eval()` before being executed with `pymongo`. This is safer and structured.

6. Error Handling and Fallback

```
except Exception as e:
      print("Error:", e)
```

General error handling is implemented here. This ensures that users are given informative messages when query generation or execution fails.

The implementation of ChatDB integrates natural language understanding, backend logic, and secure query execution into a cohesive data exploration tool. By combining GPT-4 with structured (MySQL) and semi-structured (MongoDB) data systems, the application effectively bridges the gap between non-technical users and complex database queries. The modular design—featuring backend detection, dynamic prompt engineering, and dual execution logic—demonstrates both technical robustness and user-centric flexibility. Code snippets presented above illustrate how each component contributes to the overall functionality, from backend routing to safe parsing and error management. Together with the validated query results and system behavior, these implementation details confirm that ChatDB provides a reliable and extensible framework for language-based data interaction.

Learning Outcomes

Through the development of ChatDB, we gained deep experience in full-stack system design that combines AI-driven interfaces with traditional and modern database systems. We learned how to integrate large language models like GPT-4 into real-world applications, and more importantly, how to craft effective system prompts that guide GPT's behavior based on schema and intent. We became proficient in handling safe parsing of GPT-generated outputs and learned to distinguish between unstructured model output and executable code. On the database side, we developed insights into designing hybrid systems—structured MySQL tables with relational integrity and flexible, document-oriented MongoDB schemas. We also built confidence in deploying and debugging query execution pipelines. Furthermore, we practiced version control, modular

Python development, and cross-component integration. From a team perspective, we honed our collaboration skills by combining independently built components into a single, unified product.

Challenges Faced

We faced several challenges, including tuning GPT-4 prompts to return clean and correct syntax, avoiding the use of eval() by substituting ast.literal_eval() in MongoDB, handling edge cases in backend detection logic, normalizing and loading large CSV datasets into both MySQL and MongoDB, and ensuring consistent schema mapping while avoiding field mismatches. Additionally, we had to contend with GPT-4's potential to hallucinate nonexistent fields or produce ambiguous logic. To address this, we implemented simple error detection and fallback mechanisms.

One of the most significant challenges was merging our separately developed systems—Doris's MongoDB backend and Tianqi's MySQL interface—into a unified ChatDB application. This required aligning schemas and execution logic, reconciling different prompt engineering strategies, unifying error handling methods, and implementing shared output formatting. Bridging these differences while preserving the unique logic of each backend was technically demanding and required careful coordination and modular restructuring of our shared Python codebase.

Individual Contribution

Doris designed and implemented the MongoDB backend, created the MongoDB system prompt and parsing logic, wrote load_mongo.py to split and insert documents into three collections (wages_overall, wages_gender, and wages_race), and implemented complex aggregation and projection queries for MongoDB. Tianqi designed and normalized the MySQL schema, developed the data ingestion logic via load_mysql.py, authored the SQL-specific system prompt, built query execution and result formatting modules, and implemented group by, joins, and modification commands. Both team members collaborated closely on integrating backend detection, coordinating system prompt formatting, and testing and debugging query execution within chatdb_app.py.

Conclusion

ChatDB successfully demonstrates an end-to-end natural language interface over hybrid databases. The project supports flexible, complex queries across MySQL and MongoDB using GPT-4, giving users an intuitive way to explore wage-related data. It abstracts technical complexity and enables non-expert users to interact with structured and semi-structured data alike. Our system illustrates the potential of combining generative AI with data infrastructure to lower the barrier to data-driven insights. Through modular architecture, effective prompt engineering, and careful query execution, ChatDB offers a practical solution for intelligent, language-based data exploration.

Future Scope

In the future, we plan to extend ChatDB with a user-friendly web frontend using Streamlit or Flask to make the interface accessible beyond the command line. We aim to replace keyword-based backend detection with a vector similarity model or trainable classifier to improve query routing accuracy. Additionally, we see value in building a custom domain-tuned LLM that understands policy- and wage-specific terminology for more accurate query transformation. Expanding the datasets to include additional dimensions such as industry, occupation, or geographic region will further enrich user insights. Finally, we plan to add query logging and feedback mechanisms to evaluate and continuously refine query quality, model prompting, and user experience.